

Tutorial Vue.js dari Nol

Studi Kasus: Onlineshop Sekolah (Landing, Product, About) dengan Kartu Produk

Edisi bertahap & bahasa sesederhana mungkin — dari instalasi sampai project jalan, dicoba satu per satu di browser sebelum lanjut.

0. Sebelum Mulai — Ini Bukan Hal Baru!

Kalian sudah pernah bikin "Website Sekolah" pakai JavaScript murni: ada Component.js, Navbar.js, Footer.js, Page.js, router.js, semua dipisah per file dan folder. Nah, semua itu sebenarnya LATIHAN supaya paham konsep Vue SEBELUM pakai Vue beneran.

Sekarang kita pakai Vue.js beneran. Konsepnya SAMA PERSIS, cuma alat dan penulisannya beda:

Dulu (JavaScript murni)	Sekarang (Vue.js)
components/Navbar.js (class, extends Component)	components/Navbar.vue (1 file .vue)
render() { return `...html...` }	bagian <template> di file .vue
this.props.namaSekolah	defineProps() di bagian <script setup>
components/Page.js (bungkus Navbar+Footer+isi)	App.vue (bungkus Navbar+Footer+RouterView)
router.js (kita tulis manual)	src/router/index.js (pakai vue-router)
app.js (saklar utama)	src/main.js (saklar utama)
	<RouterLink to="/about">

Project yang akan kita bangun: Onlineshop Sekolah, 3 halaman (Landing, Product, About), dan halaman Product menampilkan produk dalam bentuk kartu (card) — mirip kartu produk di Tokopedia/Shopee, cuma versi sederhana.

1. Instalasi Alat yang Dibutuhkan

Bahasa Sederhananya

Node.js itu kayak "mesin" supaya JavaScript bisa jalan di komputer kita, bukan cuma di browser. npm itu kayak "toko aplikasi" buat kode JavaScript — tempat kita unduh potongan kode orang lain biar tidak bikin dari nol.

Vite itu alat yang membantu project Vue kita jalan cepat pas development (coding) dan pas nanti mau dikirim ke internet (production).

1. Unduh dan install Node.js versi LTS dari nodejs.org (pilih yang tulisannya "LTS", bukan "Current").
2. Setelah selesai install, buka Command Prompt / Terminal, ketik dua perintah ini satu-satu untuk memastikan sudah terpasang:

```
node -v
npm -v
```

Kalau muncul nomor versi (misalnya v20.11.0 dan 10.2.4), berarti sudah siap. Kalau muncul tulisan "command not found", coba install ulang Node.js dan restart komputer.

3. Install VS Code dari code.visualstudio.com (kalau belum ada).
4. Buka VS Code, pasang extension bernama "Vue - Official" lewat menu Extensions (ikon kotak-kotak di sisi kiri) — supaya warna kode .vue muncul rapi.

2. Tahap 0: Membuat Project Vue Baru

TAHAP 0 • SETUP AWAL

Buka Terminal (atau folder tempat kalian biasa menyimpan project), lalu ketik:

```
npm create vue@latest
```

Ini akan menanyakan beberapa hal. Jawab seperti ini (ketik nama lalu Enter, atau pakai panah atas/bawah lalu Enter untuk Yes/No):

Pertanyaan	Jawaban
Project name:	onlineshop-vue
Add TypeScript?	No
Add JSX Support?	No
Add Vue Router for Single Page Application development?	Yes (WAJIB Yes, ini yang kita pakai buat routing)
Add Pinia for state management?	No
Add Vitest for Unit Testing?	No
Add an End-to-End Testing Solution?	No
Add ESLint for code quality?	No

Setelah selesai, jalankan 3 perintah ini satu-satu (ikuti juga instruksi yang muncul di terminal, biasanya persis seperti ini):

```
cd onlineshop-vue
npm install
npm run dev
```

Coba jalankan sekarang

Setelah npm run dev, terminal akan menampilkan sebuah alamat, biasanya <http://localhost:5173>. Buka alamat itu di browser (Ctrl+klik biasanya langsung buka, atau copy-paste manual).

HASIL DI BROWSER

<http://localhost:5173>

Muncul halaman bawaan Vue (logo Vue, tulisan "You did it!", dan beberapa link).

Kalau ini sudah muncul, artinya project Vue kalian sudah menyala. Jangan ditutup terminalnya — biarkan npm run dev tetap jalan selama kita koding, browser akan otomatis refresh sendiri tiap kali file disimpan.

3. Tahap 1: Kenalan dengan Struktur Folder

TAHAP 1 · STRUKTUR PROJECT

Buka folder onlineshop-vue di VS Code (File > Open Folder). Kalian akan lihat struktur seperti ini (beberapa file bawaan sudah dihapus supaya bersih):

```
onlineshop-vue/
├── index.html
├── package.json
├── src/
│   ├── main.js           <- saklar utama, mirip app.js dulu
│   ├── App.vue           <- "gedung utama", bungkus Navbar+halaman+Footer
│   ├── router/
│   │   └── index.js      <- mirip router.js dulu, tapi pakai vue-router
│   ├── components/      <- tempat Navbar.vue, Footer.vue, ProductCard.vue
│   └── pages/            <- tempat LandingPage.vue, ProductPage.vue,
AboutPage.vue (kita bikin sendiri)
```

Bahasa Sederhananya

index.html itu wadah kosongnya, sama kayak dulu — ada `<div id="app"></div>`.

main.js itu yang nyalain semuanya, sama kayak *app.js* dulu.

App.vue itu yang nampilin Navbar + isi halaman + Footer bareng-bareng, mirip *Page.js* dulu.

File yang namanya diakhiri ".vue" itu SATU FILE berisi 3 bagian: HTML-nya (`<template>`), JavaScript-nya (`<script setup>`), dan CSS-nya (`<style>`) — semua digabung jadi 1, tidak perlu dipisah kayak dulu.

Bentuk dasar 1 file .vue selalu seperti ini:

```
<template>
  <!-- HTML-nya di sini -->
</template>

<script setup>
  // JavaScript-nya di sini
</script>

<style scoped>
  /* CSS khusus buat component ini saja, tidak bocor ke component lain */
</style>
```

Buat folder baru `src/pages/` (kosong dulu, isinya kita buat di Tahap 4).

4. Tahap 2: Component Pertama — Navbar.vue

TAHAP 2 · COMPONENT PERTAMA

Buat file `src/components/Navbar.vue`:

```
src/components/Navbar.vue
<template>
  <nav>
    <h2>SMK Yadika Soreang - Onlineshop</h2>
    <RouterLink to="/">Landing</RouterLink> |
```

```
<RouterLink to="/product">Product</RouterLink> |  
<RouterLink to="/about">About</RouterLink>  
</nav>  
<hr />  
</template>
```

Bahasa Sederhananya

RouterLink itu gantinya yang dulu kita tulis manual. Bedanya, Vue OTOMATIS kasih class aktif ke link yang lagi dibuka — dulu kita harus bikin sendiri fungsi linkClass(), sekarang tidak perlu lagi!

Sekarang colokkan sementara ke App.vue supaya kelihatan hasilnya. Buka src/App.vue, ganti semua isinya jadi:

```
src/App.vue (uji coba Navbar saja)  
<script setup>  
import Navbar from './components/Navbar.vue'  
</script>  
  
<template>  
  <Navbar />  
</template>
```

Coba jalankan sekarang

Simpan kedua file. Karena npm run dev masih jalan, browser akan refresh sendiri. Kalian harus lihat judul toko dan 3 link menu. Boleh coba klik link Product/About — halamannya belum ada isinya, itu wajar, kita baru bikin Navbar saja.

5. Tahap 3: Component Kedua — Footer.vue

TAHAP 3 · COMPONENT KEDUA

```
src/components/Footer.vue  
<script setup>  
const tahun = new Date().getFullYear()  
</script>  
  
<template>  
  <hr />  
  <footer>  
    <small>&copy; {{ tahun }} SMK Yadika Soreang – Onlineshop  
Sekolah</small>  
  </footer>  
</template>
```

Bahasa Sederhananya

Tanda {{ tahun }} itu gantinya \${tahun} yang dulu kita pakai di template string. Cara bacanya: "taruh nilai variabel tahun di sini".

Update App.vue untuk uji Navbar + Footer bersamaan:

```
src/App.vue (uji coba Navbar + Footer)
```

```
<script setup>
import Navbar from './components/Navbar.vue'
import Footer from './components/Footer.vue'
</script>

<template>
  <Navbar />
  <Footer />
</template>
```

Coba jalankan sekarang

Refresh otomatis, sekarang harus muncul Navbar di atas dan tulisan copyright di bawah dengan tahun berjalan.

6. Tahap 4: Membuat 3 Halaman (masih kosong dulu)

TAHAP 4 • HALAMAN KOSONG

Buat 3 file di folder `src/pages/` — isinya sederhana dulu, nanti halaman Product kita lengkapi di Tahap 8:

`src/pages/LandingPage.vue`

```
<template>
  <main>
    <h1>Selamat Datang di Onlineshop Sekolah</h1>
    <p>Belanja gampang, sekolah senang.</p>
  </main>
</template>
```

`src/pages/ProductPage.vue`

```
<template>
  <main>
    <h1>Produk Kami</h1>
    <p>(kartu produk akan kita tambahkan di Tahap 8)</p>
  </main>
</template>
```

`src/pages/AboutPage.vue`

```
<template>
  <main>
    <h1>Tentang Kami</h1>
    <p>Onlineshop ini dikelola oleh siswa SMK Yadika Soreang jurusan PPLG.</p>
  </main>
</template>
```

Belum bisa dites langsung — halaman ini baru bisa muncul kalau sudah didaftarkan ke router. Itu tugas kita di Tahap 5.

7. Tahap 5: Mengatur Router

TAHAP 5 • PETA HALAMAN

Buka `src/router/index.js` (file ini sudah otomatis dibuat waktu kita pilih "Vue Router: Yes" di Tahap 0). Ganti isinya jadi:

```
src/router/index.js
import { createRouter, createWebHistory } from 'vue-router'
import LandingPage from '../pages/LandingPage.vue'
import ProductPage from '../pages/ProductPage.vue'
import AboutPage from '../pages/AboutPage.vue'

const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: LandingPage },
    { path: '/product', component: ProductPage },
    { path: '/about', component: AboutPage },
  ],
})

export default router
```

Bahasa Sederhananya

Ingat dulu kalian nulis sendiri objek `routes = { "/home": HomePage, ... }` dan fungsi `router()` yang baca `window.location.hash`? Nah `createRouter` dan `createWebHistory` ini Vue yang SUDAH BIKININ semua itu buat kita. Kita tinggal isi daftar path-nya saja, sama kayak buku catatan satpam yang kita bahas dulu.

8. Tahap 6: Menyambungkan Semuanya

TAHAP 6 · PENYATUAN

Buka `src/main.js`, pastikan isinya seperti ini (biasanya sudah otomatis begini):

```
src/main.js
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

const app = createApp(App)
app.use(router)
app.mount('#app')
```

Bahasa Sederhananya

`app.mount('#app')` itu versi Vue dari baris `document.getElementById('app').innerHTML = ...` yang dulu kita tulis manual di baris terakhir `app.js`.

Sekarang, kembalikan `App.vue` ke versi FINAL — bukan lagi uji coba manual, tapi memakai `RouterView` supaya halaman berganti otomatis sesuai URL:

```
src/App.vue (FINAL)
<script setup>
import Navbar from './components/Navbar.vue'
import Footer from './components/Footer.vue'
</script>
```

```
<template>
  <Navbar />
  <RouterView />
  <Footer />
</template>
```

Bahasa Sederhananya

`<RouterView />` itu "bingkai foto kosong" yang OTOMATIS diisi halaman yang cocok sama URL — gantinya `document.getElementById("app").innerHTML = page.render()` yang dulu ditulis manual di `router.js`.

Dan `App.vue` yang isinya `Navbar + RouterView + Footer` ini POLANYA SAMA PERSIS kayak `Page.js` yang dulu kita buat: bungkus `Navbar`, isi halaman, `Footer`. Cuma sekarang ditulis 1 kali di `App.vue`, otomatis dipakai buat semua halaman.

Coba jalankan sekarang (uji akhir)

Refresh browser, klik menu `Landing`, `Product`, `About` satu-satu. Isi `<main>` harus berganti tanpa reload, `Navbar & Footer` tetap selalu muncul, dan URL di address bar ikut berubah (misalnya jadi `...5173/product`).

9. Tahap 7: Component Kartu Produk (ProductCard.vue)

TAHAP 7 · COMPONENT YANG BISA DIPAKAI ULANG

Buat file `src/components/ProductCard.vue`:

```
src/components/ProductCard.vue
<script setup>
defineProps(['nama', 'harga', 'gambar'])
</script>

<template>
  <div class="card">
    
    <h3>{{ nama }}</h3>
    <p>Rp {{ harga.toLocaleString('id-ID') }}</p>
  </div>
</template>

<style scoped>
.card {
  border: 1px solid #ddd;
  border-radius: 10px;
  padding: 14px;
  width: 200px;
  text-align: center;
}
.card img {
  width: 100%;
  border-radius: 6px;
}
</style>
```

Bahasa Sederhananya

`defineProps(['nama', 'harga', 'gambar'])` itu gantinya `this.props` dulu. Artinya: "card ini akan dikirim 3 data dari luar: nama, harga, gambar".
`:src="gambar"` itu cara Vue mengisi atribut HTML pakai variabel JavaScript — tanda titik dua di depan nama atributnya artinya "isinya bukan teks biasa, tapi nilai dari JavaScript".

10. Tahap 8: Menampilkan Banyak Kartu dengan Perulangan

TAHAP 8 · DAFTAR PRODUK

Sekarang lengkapi `src/pages/ProductPage.vue` supaya menampilkan beberapa `ProductCard` sekaligus dari sebuah daftar data:

```
src/pages/ProductPage.vue
<script setup>
import ProductCard from '../components/ProductCard.vue'

const daftarProduk = [
  { id: 1, nama: 'Kaos Sekolah', harga: 75000, gambar:
'https://placeholder.co/150' },
  { id: 2, nama: 'Topi Sekolah', harga: 35000, gambar:
'https://placeholder.co/150' },
  { id: 3, nama: 'Tas Sekolah', harga: 120000, gambar:
'https://placeholder.co/150' },
]
</script>

<template>
  <main>
    <h1>Produk Kami</h1>
    <div class="grid">
      <ProductCard
        v-for="produk in daftarProduk"
        :key="produk.id"
        :nama="produk.nama"
        :harga="produk.harga"
        :gambar="produk.gambar"
      />
    </div>
  </main>
</template>

<style scoped>
.grid {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
}
</style>
```

Bahasa Sederhananya

`v-for="produk in daftarProduk"` itu gantinya `for` (`const` halaman of `daftarHalaman`) yang dulu kita tulis di JavaScript murni — bedanya sekarang ditulis LANGSUNG di HTML-nya, dan Vue otomatis mengulang `<ProductCard>` untuk setiap produk di `daftarProduk`.

`:key="produk.id"` itu WAJIB ditulis tiap kali pakai `v-for` — gunanya supaya Vue bisa membedakan kartu satu dengan yang lain (dia butuh "nomor identitas" yang unik untuk tiap kartu).

Coba jalankan sekarang

Refresh browser, klik menu Product. Harus muncul 3 kartu produk berjajar, masing-masing dengan gambar, nama, dan harga yang berbeda — padahal cuma menulis `<ProductCard />` satu kali di kode.

HASIL DI BROWSER

`http://localhost:5173/product`

[Kaos Sekolah] [Topi Sekolah] [Tas Sekolah]

Rp 75.000

Rp 35.000

Rp 120.000

Alur Lengkap Project Ini

```
index.html
├── src/main.js
│   ├── src/App.vue
│   │   ├── components/Navbar.vue
│   │   ├── components/Footer.vue
│   │   └── <RouterView /> -> diisi salah satu halaman di bawah, sesuai URL
│   └── src/router/index.js
│       ├── pages/LandingPage.vue
│       ├── pages/ProductPage.vue
│       │   └── components/ProductCard.vue (dipanggil berkali-kali lewat v-
for)
│       └── pages/AboutPage.vue
```

Penutup

Bandingkan lagi dengan yang dulu kalian pelajari — semua istilah baru ini sebenarnya "nama resmi" dari hal yang sudah pernah kalian buat sendiri secara manual:

- Component (Navbar.js, Footer.js dulu) → sekarang jadi file .vue, isinya digabung template + script + style.
- props (this.props dulu) → sekarang defineProps().
- Template string `${...}` dulu → sekarang `{{ ... }}` di dalam `<template>`.
- router.js manual dulu (baca hash, cocokkan path) → sekarang `createRouter()` dari vue-router, otomatis semua.
- Page.js (bungkus Navbar+isi+Footer) dulu → sekarang App.vue dengan `<RouterView />`.
- `for...of` manual dulu buat render banyak halaman/kartu → sekarang `v-for` langsung di HTML.

Kalau sudah nyaman dengan alur ini, project onlineshop-vue ini bisa terus dikembangkan: tambah halaman Keranjang, tambah filter kategori produk, atau ambil data produk dari API sungguhan — tapi pola dasarnya akan selalu sama seperti yang baru saja kalian bangun. Selamat mencoba!